QR Notifier System, QR-Alimy Website, and Real-Time Chat Application

An internship

Submitted in partial fulfillment of the requirements
for the award of the Degree of
Integrated Master of Science

IN

QUANTITATIVE ECONOMICS AND DATA SCIENCE

BY

UJJAWAL SAURAV

IED/10016/21

BIRLA INSTITUTE OF TECHNOLOGY
MESRA-835215, RANCHI

APPROVAL OF THE GUIDE

Recommended that the internship report entitled **"QR Notifier System, QR-Alimy Website, and Real-Time Chat Application"** presented by **Ujjawal Saurav** under my supervision and guidance be accepted as fulfilling the requirements for the completion of the six-month internship programme. The report combines carried out during the internship period from January 2026 to June 2026. To the best of my knowledge, the content of this report has not been submitted for the award of any previous degree, diploma, or internship certificate at any other institute.

Date: 08/05/2026

Dr. Mrinal Jana
Assistant Professor
Department of QEDS
Birla Institute of Technology
Mesra, Ranchi

DECLARATION CERTIFICATE

I hereby declare that:

a) The work presented in this internship report is the result of my own effort, carried out under the supervision of my project guide.

b) This work has not been submitted, either in part or in full, to any other institute or organization for the award of any previous degree or diploma.

c) I have followed the guidelines issued by the Institute for writing and formatting the report.

d) I have adhered to the norms and guidelines laid down in the Ethical Code of Conduct of the Institute.

e) Whenever material (data, theoretical arguments, or text) from other sources has been used, it has been acknowledged appropriately in the text and fully cited in the references.

f) Whenever I have quoted directly from other written sources, I have placed such text within quotation marks and provided proper citation and reference details.

UJJAWAL SAURAV
(IED/10016/21)

CERTIFICATE OF APPROVAL

This is to certify that the thesis entitled **"QR Notifier System, QR-Alimy Website, and Real-Time Chat Application"** submitted by **UJJAWAL SAURAV (IED/10016/21)** has been examined and approved for the award of the degree of **INTEGRATED MASTER OF SCIENCE IN QUANTITATIVE ECONOMICS AND DATA SCIENCE** of Birla Institute of Technology, Mesra, Ranchi.

Date: 08/05/2026
Place: Mesra, Ranchi

Internal Examiner **External Examiner**

(Chairperson)
Head of Department

ABSTRACT

This document presents a comprehensive technical account of the software engineering and web development contributions made during a six-month remote internship at **JEJECOMMS Co., Ltd.**, South Korea, from January 3, 2026, to June 22, 2026. Functioning as a Frontend Developer under the direct supervision of Founder and CEO Mr. Kim Joon Gang, the internship bridged theoretical computer science paradigms with agile, production-grade software development. The internship was divided into three core deliverables, each addressing specific privacy and communication challenges in urban infrastructure. A platform architected to eliminate the privacy hazard of leaving personal mobile numbers on car dashboards. Built using a modern JavaScript stack comprising React 18, TypeScript, Tailwind CSS, and Vite, the frontend establishes a tokenized secure bridge, allowing users to scan a QR code and anonymously notify a vehicle owner without exposing either party's identifiable information. The meticulous development of the *QR-Alimy Promotional Website*. Utilizing raw HTML5, CSS3, JavaScript (ES6+), jQuery, and PHP for lightweight server routing, this task involved translating high-fidelity Figma designs into responsive, cross-device compatible web pages. Critical engineering challenges addressed in this phase include implementing dynamic image mapping for interactive user guides, mitigating cumulative layout shifts (CLS), and executing complex Document Object Model (DOM) manipulations without relying on heavy frameworks. The design, logic, and deployment of a *Real-Time Chat Application*. This module explores the deep integration of Google Firebase Cloud Services (Authentication, Cloud Firestore, and Firebase Storage) with a custom HTML/CSS/JS frontend. It provides an extensive breakdown of NoSQL document architectures, real-time data synchronization utilizing WebSocket-based `onSnapshot` listeners, asynchronous media compression and uploads, typing indicators via debouncing mechanisms, message pagination using cursor-based querying, and the implementation of stringent, server-evaluated Firestore security rules.

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to my internal project guide, **Dr. Mrinal Jana**, Assistant Professor, QEDS, for his continuous guidance, patience, and support throughout the period of this internship. Interacting with him has helped me develop a clearer understanding of software engineering fundamentals and encouraged me to implement rigorous engineering practices. His academic feedback at various stages of the project has been invaluable in shaping this report.

I am equally thankful to **Dr. Vandana Bhattacharjee**, Head of Department, QEDS, BIT Mesra, for providing the academic environment, infrastructure, and administrative support that made this work possible. I also acknowledge all the faculty members of the QEDS department for their teaching and continuous encouragement.

My sincere thanks go to **Mr. Kim Joon Gang**, Founder and CEO of **JEJECOMMS Co., Ltd.**, and the entire technical team based at the Seongnam Global Convergence Center. Providing me with this remote internship opportunity allowed me to work on impactful projects like JeJeComm's Shift and the QR Alimy service. Their mentorship in writing clean code, adhering to strict Figma design constraints, and managing real-world software deployments gave me a vital foundation in industry-standard web development.

Above all, I owe a special debt to my parents for their constant encouragement, emotional support, and belief in me over the years. Their sacrifices and motivation have been the backbone of my academic journey.

Thank you.

DATE: 08/05/2026

Ujjawal Saurav
(IED/10016/21)

Contents

List of Figures x

List of Tables xi

1 Introduction 1

1.1 Overview of the Internship 1

1.2 About the Organization 1

1.3 Motivation and Problem Statement 2

1.4 Objectives of the Internship 2

2 System Analysis and Feasibility Study 4

2.1 Introduction 4

2.2 Requirement Gathering 4

2.2.1 Functional Requirements 4

2.2.2 Non-Functional Requirements 5

2.3 Feasibility Study 6

2.3.1 Technical Feasibility 6

2.3.2 Economic Feasibility 6

2.3.3 Operational Feasibility 6

2.4 Software Development Life Cycle (SDLC) 7

3 Technology Stack 8

3.1 Introduction 8

3.2 Frontend Markup and Styling Technologies 8

3.2.1 HTML5 Semantic Architecture 8

3.2.2 CSS3 and Responsive Web Design (RWD) 8

3.2.3 Tailwind CSS and Utility-First Styling 9

3.3 Client-Side Scripting and Frameworks 9

3.3.1 JavaScript (ES6+) 9

3.3.2 jQuery Library 10

3.3.3 React 18 and TypeScript (Task 1) 10

3.4 Backend-as-a-Service (BaaS) Infrastructure 10

3.4.1 Google Firebase Cloud Platform 10

3.5 Development and Build Tools 11

3.5.1 Vite 11

3.5.2 Git, GitHub, and Figma 11

4 Task 1: QR Notifier System Development 12

4.1 Introduction to Task 1 12
4.2 System Image & User Interface 12
4.3 System Architecture and Design 13
4.3.1 Directory Structure and Separation of Concerns 13
4.4 Core Implementation Details 14
4.4.1 Application Entry and DOM Mounting 14
4.4.2 Routing Configuration 15
4.4.3 State Management for Interactive Components 15
4.4.4 Styling and Responsive Constraints with Tailwind CSS 17
4.5 The QR Communication Workflow Design 18
4.5.1 Stage 1: Vehicle Registration 19
4.5.2 Stage 2: QR Tokenization 19
4.5.3 Stage 3: Secure Scanning Protocol 19
4.5.4 Stage 4: Notification Dispatch 19
4.6 Challenges and Solutions in Task 1 20

5 Task 2: QR-Alimy Promotional Website 21

5.1 Introduction to Task 2 21
5.2 Project Architecture and File Organization 21
5.3 Detailed Feature Implementation 22
5.3.1 Responsive CSS Architecture via Media Segmentation 22
5.4 QR Registration Workflow and Sticker Placement 22
5.4.1 QR Registration Workflow 22
5.4.2 Vehicle QR Sticker Placement 23
5.4.3 Improved QR Workflow UI Architecture 24
5.4.4 Interactive User Guide & Dynamic Image Mapping 25

6 Task 3: Real-Time Chat Application with Firebase 27

6.1 Introduction to Task 3 27
6.2 System Interface and Live Application 27
6.3 System Architecture and Serverless Model 29
6.4 Database Design: NoSQL Document Schema 29
6.5 Core Functionalities Implementation 30
6.5.1 Anonymous Authentication Pipeline 30
6.5.2 Real-Time Message Synchronization via WebSockets 31
6.5.3 Image Upload Pipeline and Compression 32

6.6 Advanced Algorithms Developed 34
6.6.1 Typing Indicators and Function Debouncing 34
6.6.2 Cursor-Based Pagination for Message History 34
6.7 Firestore Security Rules Architecture 35

7 Software Testing and Quality Assurance **37**
7.1 Overview of Testing Methodologies 37
7.2 Manual UI and Cross-Browser Emulation 37
7.3 Functional Testing of the Chat Engine 37
7.4 Automated Firebase Emulator Testing 38

8 Conclusion and Future Scope **41**
8.1 Conclusion 41
8.2 Future Scope 42

A Local Development and Deployment Instructions **44**
A.1 A.1 Running Task 1 (React SPA via Vite) 44
A.2 A.2 Running Task 2 (PHP Static Site Routing) 45
A.3 A.3 Running Task 3 (Firebase Chat Architecture) 45

B Glossary of Technical Terminology **47**

REFERENCES **49**

List of Figures

4.1 QR Alimi project image from the repository demonstrating the frontend layout. 13
4.2 System Workflow visualization model integrated into the QR Notifier landing page 19

5.1 QR registration and onboarding workflow 23
5.2 Recommended QR sticker positioning for different vehicle types 24
5.3 Improved workflow architecture and frontend layout design 25

6.1 Chat Room interface — passive user view. Incoming messages appear left-aligned with timestamps; date separators partition the message history by calendar day. 28
6.2 Chat Room interface — active sender view. The authenticated user's own messages are rendered right-aligned with green bubbles, while the other participant's message remains left-aligned. 28
6.3 Real-time chat application serverless architecture workflow 29

List of Tables

7.1 Heuristic Test Cases for Real-Time Chat Functionality 38

B.1 Glossary of Software Engineering Terms Used 47

Chapter 1

Introduction

1.1 Overview of the Internship

The landscape of web development and software engineering has undergone a massive paradigm shift in the last decade. Applications have evolved from statically served HTML documents to highly interactive, cloud-backed Single Page Applications (SPAs) that require real-time bidirectional data flow. To bridge the gap between theoretical knowledge acquired during academic studies and the practical, agile demands of the modern software industry, a comprehensive six-month industrial internship was undertaken at **JEJECOMMS Co., Ltd.** from January 3, 2026, to June 22, 2026.

This report provides a granular, highly technical account of the software engineering tasks completed during this tenure. The role assigned was ”Frontend Web Developer,” and the work encompassed a vast spectrum of web technologies—from raw, vanilla scripting languages to modern, state-driven JavaScript frameworks. The internship demanded a daily commitment of 4 to 8 hours, requiring rigorous remote coordination, version control management, and continuous integration with the development team based in South Korea.

1.2 About the Organization

JEJECOMMS Co., Ltd. is an innovative technology organization situated at the Seongnam Global Convergence Center in Sujeong-gu, South Korea. The company specializes in creating secure, privacy-first communication and mobility tools. Their flagship product is the **QR Alimy** service. QR Alimy operates as a smart parking notification system designed to prevent the public exposure of personal mobile numbers by utilizing tokenized QR codes. The organization strictly adheres to international data protection laws, emphasizing end-to-end user privacy, scalable backend infrastructure, and seamless, intuitive User Interface (UI) and User Experience (UX) designs.

1.3 Motivation and Problem Statement

In dense urban environments across the globe, limited parking infrastructure frequently results in vehicles temporarily blocking each other in residential complexes, commercial zones, and public lots. The prevailing societal workaround for this issue is for drivers to physically display their personal mobile phone numbers on a placard or business card placed on their vehicle's dashboard.

However, this traditional method creates a severe privacy vulnerability. Malicious actors, data scrapers, or passersby can easily harvest these numbers. This widespread exposure leads to severe consequences, including targeted SMS spam, phishing attacks, identity theft, and personal harassment.

The core motivation behind the tasks assigned during this internship was to architect and build digital interfaces that solve this exact problem. By routing communication through a secure, anonymous cloud server triggered via a smartphone QR scan, vehicle owners can be contacted securely without their actual phone number ever being transmitted to the scanner's device. Building the robust frontend infrastructure to support this vision required developing highly responsive websites and real-time communication modules capable of handling high concurrency and strict privacy rules.

1.4 Objectives of the Internship

The specific technical, analytical, and professional objectives set forth for this six-month development period included:

1. **Mastery of Frontend Frameworks:** To gain deep, practical proficiency in React 18 [7], TypeScript [3], and Vite [10] by building robust, component-based Single Page Architectures (SPAs) that manage complex internal state without unnecessary DOM repaints.

2. **UI/UX Translation and Implementation:** To meticulously translate high-fidelity Figma mockups into pixel-perfect HTML/CSS layouts, utilizing both custom cascading stylesheets and the Tailwind CSS [9] utility-first framework, ensuring responsive behavior across all viewport dimensions.

3. **Cloud Integration and Serverless Architecture:** To understand Backend-as-a-Service (BaaS) architectures by integrating Google Firebase [2] (Authentication, Cloud Firestore, and Firebase Storage) into a real-time chat application, establishing WebSocket connections for live data syncing.

4. **Legacy Codebase Management and Optimization:** To interact with, refactor, and improve traditional web stacks involving jQuery [5], raw ES6+ JavaScript, and PHP [6] routing, maintaining backward compatibility while optimizing load times and asset delivery.

5. **Software Development Lifecycle (SDLC) Execution:** To actively participate in agile software development processes, including daily stand-ups via Slack, rigorous code reviews, systematic debugging methodologies, and version control branching strategies via GitHub.

Chapter 2

System Analysis and Feasibility Study

2.1 Introduction

Before the initiation of any coding or architectural design, a thorough system analysis was conducted to understand the requirements of the QR Alimy system and the Real-Time Chat platform. This phase ensures that the proposed software solution aligns with business goals, technical constraints, and user needs. The analysis also explores the viability of the project from multiple perspectives to mitigate risks early in the development lifecycle.

2.2 Requirement Gathering

The requirements for the projects were divided into functional and non-functional categories. Functional requirements define what the system must do, while non-functional requirements define how the system must perform or the constraints under which it must operate.

2.2.1 Functional Requirements

The core deliverables were specified with the following mandatory functionalities:

a) **QR Notifier (React/TS Stack):**

- The system must provide a landing page showcasing the service's key benefits and workflow.
- Users must be able to register vehicle details and their preferred anonymous contact token.
- The application must generate a unique, non-identifiable QR code linked to the owner's encrypted token.
- The system must implement a "Scan Page" that allows scanners to select pre-configured messages (e.g., "Please move car") without exposing their own information.

b) **Promotional Website (Static Stack):**

- The website must offer a high-fidelity guide section demonstrating app installation and usage.
- The interface must include interactive, dynamic visual components (e.g., carousels and image maps) to improve user engagement.
- The site must link directly to the App Store and Google Play Store for the main QR Alimy mobile application.

c) **Real-Time Chat (HTML/JS + Firebase Stack):**

- Users must be able to join chat rooms and authenticate anonymously using Firebase Auth [2].
- The chat module must support text and image messaging.
- Image attachments must not exceed 5MB.
- The UI must display real-time typing indicators (”User is typing...”).
- The application must track online presence status (Online/Offline) for all members.
- The system must implement cursor-based pagination to load message history efficiently when scrolling up.

2.2.2 Non-Functional Requirements

Non-functional requirements were established to ensure high performance, security, and exceptional user experience across diverse environments:

- **Performance (LCP):** Web pages must achieve a Largest Contentful Paint (LCP) of under 2.5 seconds on mobile 3G networks.
- **Security (Rules):** Database access must be strictly controlled; users cannot read or write messages outside their authorized channels. Firebase Firestore Security Rules [2] must mathematically enforce these constraints.
- **Usability (Lighthouse):** The UI must strictly follow the provided Figma designs, achieving an accessibility and performance score of at least 90 on Google Lighthouse audits.
- **Scalability (Firebase):** The chat application architecture must support hundreds of concurrent users without degrading message delivery latency.

2.3 Feasibility Study

A feasibility study was conducted to evaluate the viability of the project across three main parameters: technical, economic, and operational.

2.3.1 Technical Feasibility

The project demands the integration of reactive frontends with real-time backend synchronization.

- Using React 18 [7] for the frontend is technically sound given its widespread community support, robust virtual DOM, and component reusability.
- Utilizing Firebase [2] as the backend eliminates the need for maintaining dedicated physical servers. Firebase's SDK natively handles WebSockets (via `onSnapshot`), overcoming the immense technical hurdle of building a real-time polling or socket engine from scratch. Thus, the project was deemed highly technically feasible.

2.3.2 Economic Feasibility

Because JEJECOMMS Co., Ltd. is a startup, optimizing operational expenditures (OpEx) is crucial.

- Open-source tools like React [7], Vite [10], Tailwind CSS [9], and Node.js require zero licensing fees.
- Firebase offers a generous "Spark" free tier, which easily covers the development, testing, and early production phases of the chat application [2]. Costs scale linearly only as user traffic increases, making it economically sound.

2.3.3 Operational Feasibility

The system aims to replace physical phone numbers with scanned QR codes. Operationally, modern smartphones (iOS and Android) have native QR code scanners built into their default camera applications. Users do not need to download a specialized third-party app simply to scan the vehicle's QR code. This lowers the friction of adoption significantly, proving the system is operationally viable for the general public.

2.4 Software Development Life Cycle (SDLC)

The development process adhered to the **Agile Methodology**. Unlike traditional Waterfall models where requirements are locked early, Agile allows for iterative development, continuous feedback, and adaptive planning.

The 6-month internship was broken down into two-week sprints.

- **Sprint Planning:** Held via Slack to determine which Figma screens or backend features would be tackled.
- **Development:** Code was written locally, utilizing Git branches (e.g., `feature/chat-upload`).
- **Code Review:** Pull requests were submitted and reviewed by the technical lead in South Korea.
- **Deployment & QA:** Merged code was pushed to staging environments for cross-browser testing.

Chapter 3

Technology Stack

3.1 Introduction

To execute the software solutions effectively across three distinct tasks, a comprehensive understanding of diverse frontend and backend technological stacks was required. The internship demanded bridging theoretical computer science paradigms with agile, production-grade software development. The technologies utilized range from raw, vanilla scripting languages to state-driven reactive frameworks and serverless cloud architectures.

3.2 Frontend Markup and Styling Technologies

3.2.1 HTML5 Semantic Architecture

HyperText Markup Language (HTML) is the foundational standard markup language for documents designed to be displayed in a web browser. The projects utilized HTML5 [4], which introduced semantic tags carrying structural meaning (e.g., `<header>`, `<nav>`, `<section>`, `<article>`, `<footer>`). Utilizing semantic HTML5 was critical for improving web accessibility (a11y) for screen readers and vastly enhancing Search Engine Optimization (SEO) algorithms.

3.2.2 CSS3 and Responsive Web Design (RWD)

Cascading Style Sheets (CSS3) manages the visual presentation and layout of the HTML DOM elements [4]. Modern CSS features utilized heavily across all tasks include:

- **CSS Flexbox (Flexible Box Layout):** A one-dimensional layout model utilized for aligning items in rows or columns. Flexbox was mathematically essential for Task 3's real-time chat, facilitating vertical centering, reverse-scrolling message containers, and input bars that scale perfectly regardless of the viewport width.
- **CSS Grid Layout:** A two-dimensional layout system used for complex grid-based layouts. Grid was heavily used on the promotional website (Task 2) to manage

complex sections containing masonry-style galleries and multi-column feature lists without relying on floating elements or complex margin calculations.

- **Media Queries:** The absolute backbone of Responsive Web Design (RWD). Media queries allow the frontend to apply different CSS rules based on the device's characteristics (primarily `max-width` and `min-width`). This ensured that the interfaces adapted gracefully to mobile, tablet, and desktop viewports, preventing horizontal scrolling and unreadable text.

3.2.3 Tailwind CSS and Utility-First Styling

While Tasks 2 and 3 utilized custom CSS stylesheets, Task 1 utilized **Tailwind CSS** [9]. Tailwind is a utility-first CSS framework packed with classes like `flex`, `pt-4`, `text-center`, and `rotate-90` that can be composed to build any design directly in the markup.

The traditional semantic CSS approach requires switching between HTML and CSS files to invent abstract class names (like `.btn-primary` or `.chat-container`). Tailwind inverses this paradigm [9]. The framework operates using a Just-In-Time (JIT) compiler that scans all source files for Tailwind class names and generates only the CSS that is actually used. This prevents CSS bloat, entirely eliminates dead CSS code from production bundles, and keeps the production stylesheet extremely small.

3.3 Client-Side Scripting and Frameworks

3.3.1 JavaScript (ES6+)

JavaScript is the high-level, often just-in-time compiled programming language that enables interactive web pages. The projects heavily utilized ECMAScript 2015 (ES6) and later features [4]. Key mechanisms utilized included:

- **Arrow Functions:** Providing concise syntax and lexically binding the `this` context.

- **Destructuring Assignment:** Extracting data from arrays or objects into distinct variables efficiently.

- **Promises and Async/Await:** Handling asynchronous operations, specifically network requests to Firebase services, ensuring non-blocking code execution. This was vital for Tasks 1 and 3 to manage network latency without freezing the main thread.

3.3.2 jQuery Library

jQuery is a fast, small, and feature-rich JavaScript library [5]. While modern JavaScript (Vanilla JS) has incorporated many of jQuery's features natively (e.g., `querySelector`), jQuery remains invaluable for interacting with legacy codebases or quickly prototyping complex DOM manipulation. In Task 2, jQuery was utilized to simplify HTML document traversal, abstract complex event delegation across varying browser implementations, and handle smooth cross-browser DOM animations (like `fadeIn()` and `fadeOut()`) for interactive user guides [5].

3.3.3 React 18 and TypeScript (Task 1)

React is an open-source front-end JavaScript library for building user interfaces based on independent, reusable UI components [7]. Developed and maintained by Meta, React utilizes a **Virtual DOM**. Instead of updating the browser's DOM directly (which is computationally expensive), React calculates the difference (the "diff") between the previous Virtual DOM and the updated Virtual DOM and then surgically updates only the changed nodes in the real DOM.

React 18 introduced concurrent rendering, improving responsiveness by allowing the rendering process to be interruptible [7]. Task 1 utilized React Functional Components heavily, relying on hooks like `useState` for local state management and `useEffect` for side-effects and lifecycle handling.

TypeScript [3], a strict syntactical superset of JavaScript developed by Microsoft, adds optional static typing to the language. Combining React with TypeScript makes the development process highly predictable. Interfaces and Types define the exact shape of data structures, catching prop-type mismatches, incorrect API responses, and structural errors during the compilation phase within the IDE [3].

3.4 Backend-as-a-Service (BaaS) Infrastructure

3.4.1 Google Firebase Cloud Platform

Firebase is a comprehensive platform developed by Google for creating mobile and web applications [2]. It provides a Backend-as-a-Service (BaaS) model. Traditional full-stack development requires provisioning servers (like AWS EC2), writing backend APIs (in Node.js, Python, etc.), and managing database connections. Firebase abstracts this entirely, providing SDKs that allow frontend applications to communicate directly and securely with the cloud. The specific services utilized in Task 3 include:

- **Firebase Authentication:** A service that manages user identity securely [2]. For the chat app, Anonymous Authentication was used. This creates a temporary, cryptographically secure JWT (JSON Web Token) for the user without requiring them to input an email or password, lowering the friction for initial app usage.

- **Cloud Firestore:** A flexible, globally scalable NoSQL cloud database [2]. Data is stored in JSON-like documents, which are organized into collections. Firestore's defining feature is its adherence to real-time data sync using WebSockets. Clients attach `onSnapshot` listeners. Instead of the client polling the database via HTTP GET requests every few seconds (which is slow and expensive), the server pushes data updates to connected clients the millisecond the database changes.

- **Firebase Storage:** Built on Google Cloud Storage, this service is designed to store and serve user-generated media content, scaling automatically to handle exabytes of data [2]. It was utilized for the chat application's image upload pipeline, handling asynchronous byte streams and generating production-ready public download URLs.

3.5 Development and Build Tools

3.5.1 Vite

Vite (French for "quick") is a modern build tool that provides a faster and leaner development experience [10]. Traditional bundlers like Webpack crawl the entire application to build a bundle before the local server can start. Vite, conversely, serves source code over native ESM (ECMAScript Modules) directly to the browser, relying on the browser to parse the modules. This results in local development server startup times of milliseconds, regardless of the application's size, and instant Hot Module Replacement (HMR) during coding [10].

3.5.2 Git, GitHub, and Figma

Git was the primary Version Control System (VCS) used for tracking changes in source code, allowing for safe experimentation through feature branching. GitHub provided the remote repository hosting for team collaboration and remote South Korea reviews. Figma, a collaborative web application for interface design, provided the high-fidelity UI/UX mockups. The primary engineering challenge was extracting CSS properties (colors, typography, spacing constraints, asset metrics) from Figma and coding them mathematically into the frontend frameworks to ensure a pixel-perfect match.

Chapter 4

Task 1: QR Notifier System Development

4.1 Introduction to Task 1

The first operative phase of the internship focused on developing the primary frontend architecture and interface for the **QR Notifier System (QR Alimi)**. As identified in the problem statement, the objective of this system is to mitigate the privacy vulnerabilities inherent in displaying physical phone numbers on parked vehicles.

The assigned task required building a robust, component-based Single Page Application (SPA) using React [7]. This frontend needed to serve a dual purpose: first, as a highly persuasive, visually appealing landing page that effectively communicated the product's unique value proposition; and second, as a structured architectural foundation prepared to seamlessly integrate with a complex RESTful backend API in the future.

4.2 System Image & User Interface

The project successfully established the product identity, privacy-focused value proposition, and the QR workflow. Below is a representation of the system interface developed during this task.

Figure 4.1: QR Alimi project image from the repository demonstrating the frontend layout.

4.3 System Architecture and Design

Developing a scalable React application requires meticulous planning of the component tree and directory structure. The frontend was designed using a modular architecture, ensuring that UI elements were decoupled from business logic.

4.3.1 Directory Structure and Separation of Concerns

The repository was organized to separate static assets, reusable UI components (buttons, badges, modals utilizing Shadcn UI [8]), page-level layouts, utility functions, and custom React hooks.

```
QR-Alimi-MainPage/
|-- public/
|    |-- vite.svg            # Favicon and base static assets
|    |-- opengraph.png       # SEO social sharing image
|    |-- robots.txt          # Web crawler configuration
|-- src/
|    |-- assets/             # Local optimized images, SVGs
|    |-- components/
|    |    |-- ui/            # Shadcn reusable primitive components
|    |    |    |-- button.tsx
|    |    |    |-- card.tsx
|    |    |    |-- badge.tsx
|    |    |    |-- toast.tsx
|    |    |-- Header.tsx     # Global navigation component
```

```
|   |   |-- HeroSection.tsx
|   |   |-- Footer.tsx
|   |-- hooks/            # Custom React hooks
|   |   |-- use-toast.ts  # Notification state management
|   |-- lib/
|   |   |-- utils.ts      # Tailwind class merging utilities
|   |-- pages/
|   |   |-- index.tsx     # Main landing page aggregation
|   |   |-- NotFound.tsx  # 404 Error page boundary
|   |-- App.tsx           # Global Routing configuration
|   |-- main.tsx          # Application entry point and DOM mounting
|   |-- index.css         # Tailwind base directives and global resets
|-- package.json          # Node dependency tracking
|-- tailwind.config.ts    # Tailwind theme extensions and breakpoints
|-- tsconfig.json         # TypeScript compiler strictness settings
|-- vite.config.ts        # Vite build optimization settings
```

Listing 4.1: Task 1: React Project Directory Structure

4.4 Core Implementation Details

4.4.1 Application Entry and DOM Mounting

The application lifecycle begins in the `src/main.tsx` file. Utilizing the new React 18 `createRoot` API [7], the virtual DOM is attached to the physical HTML DOM element with the ID of "root". The global stylesheet is imported here, injecting the Tailwind CSS layers into the application.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

// Initialize React 18 Concurrent Mode root
const rootElement = document.getElementById("root");
if (!rootElement) throw new Error("Failed to find the root element");

createRoot(rootElement).render(
  <StrictMode>
```

```
    <App />
  </StrictMode>
);
```

Listing 4.2: Application Entry — main.tsx

4.4.2 Routing Configuration

React Router DOM (v6) was utilized to handle client-side routing. Traditional websites request a new HTML document from the server on every link click. An SPA intercepts this click, changes the URL in the browser's history API, and dynamically renders the new component without a page refresh. The `App.tsx` file serves as the routing controller.

```
import { BrowserRouter as Router, Routes, Route } from
    'react-router-dom';
import Index from './pages/index';
import NotFound from './pages/NotFound';

function App() {
    return (
        <Router>
            <Routes>
                {/* Primary Landing Page Route */}
                <Route path="/" element={<Index />} />

                {/* Catch-all route for undefined paths to
                    handle 404s */}
                <Route path="*" element={<NotFound />} />
            </Routes>
        </Router>
    );
}

export default App;
```

Listing 4.3: Routing logic and Catch-All Boundary — App.tsx

4.4.3 State Management for Interactive Components

The main promotional page required dynamic interactivity to engage users. A prominent feature was the "News Carousel," which automatically cycled through various use-cases and privacy warnings regarding exposed phone numbers.

Managing this component required React's `useState` and `useEffect` hooks [7]. To prevent memory leaks, the `setInterval` timer used to auto-advance the slides was carefully cleared in the `useEffect` cleanup function.

```
import React, { useState, useEffect } from 'react';

// TypeScript interface defining the shape of a slide object
interface SlideData {
    id: number;
    headline: string;
    imageUrl: string;
}

interface CarouselProps {
    slides: SlideData[];
}

const NewsCarousel: React.FC<CarouselProps> = ({ slides }) =>
    {
    const [currentIndex, setCurrentIndex] =
        useState<number>(0);

    // Auto-advance slides every 5 seconds
    useEffect(() => {
        const timer = setInterval(() => {
            setCurrentIndex((prevIndex) => (prevIndex + 1) %
                slides.length);
        }, 5000);

        // Cleanup function prevents duplicate intervals on
            re-render
        return () => clearInterval(timer);
    }, [slides.length]);

    const goToSlide = (index: number) => {
        setCurrentIndex(index);
    };

    return (
        <div className="relative overflow-hidden w-full h-64
            bg-gray-900 rounded-xl">
            {/* Map over slides array and conditionally apply
                transition classes */}
            {slides.map((slide, index) => (
                <div
                  key={slide.id}
```

```
                    className={`absolute inset-0 transition-opacity duration-700 ease-in-out ${
                      index === currentIndex ? 'opacity-100 z-10' : 'opacity-0 z-0'
                    }`}
                  >
                      <img src={slide.imageUrl} alt="News" className="w-full h-full object-cover opacity-50" />
                      <h2 className="absolute bottom-4 left-4 text-white text-xl font-bold">
                        {slide.headline}
                      </h2>
                  </div>
              ))}

              {/* Dot Navigation Indicators */}
              <div className="absolute bottom-2 right-4 flex space-x-2 z-20">
                  {slides.map((_, idx) => (
                      <button
                        key={idx}
                        onClick={() => goToSlide(idx)}
                        className={`w-3 h-3 rounded-full ${idx === currentIndex ? 'bg-accentyellow' : 'bg-gray-400'}`}
                      />
                  ))}
              </div>
          </div>
      );
};

export default NewsCarousel;
```

Listing 4.4: Carousel State Management and Memory Leak Prevention

4.4.4 Styling and Responsive Constraints with Tailwind CSS

A core objective was to ensure the application appeared flawless across all devices. Tailwind CSS facilitated this through its mobile-first utility classes [9]. Breakpoints (e.g., `sm:`, `md:`, `lg:`) were applied directly to HTML class attributes to reflow grids and adjust typography.

For instance, the ”Feature Grid” that outlines the benefits of QR Alimi transitions mathematically from a single-column layout on mobile phones to a three-column layout on wide desktop displays.

```
<section className="py-12 px-4 max-w-7xl mx-auto">
    <h2 className="text-3xl font-bold text-center mb-8">Why Choose QR Alimi?</h2>

    {/* CSS Grid: 1 col on mobile, 2 cols on tablets, 3 cols on desktops */}
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">

        {/* Feature Card 1 */}
        <div className="bg-white p-6 rounded-lg shadow-md hover:shadow-xl transition-shadow">
            <ShieldIcon className="w-10 h-10 text-primaryblue mb-4" />
            <h3 className="text-xl font-semibold mb-2">Absolute Privacy</h3>
            <p className="text-gray-600">Your personal phone number is never exposed to the person scanning the code.</p>
        </div>

        {/* Feature Card 2 & 3 omitted for brevity... */}
    </div>
</section>
```

Listing 4.5: Responsive Grid Layout using Tailwind CSS utilities

4.5 The QR Communication Workflow Design

Although Task 1 was strictly focused on frontend implementation, the UI had to systematically educate the user on the logical workflow of the QR Alimi product. The system operates on a four-stage interaction model, which was mapped out visually on the landing page using vector iconography.

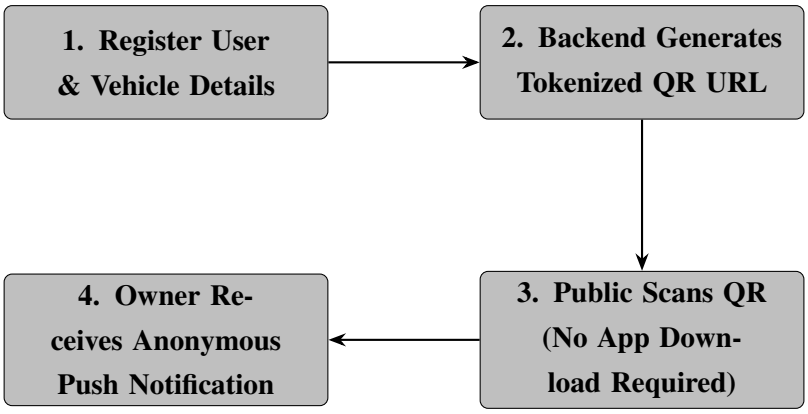

Figure 4.2: System Workflow visualization model integrated into the QR Notifier landing page

4.5.1 Stage 1: Vehicle Registration

The vehicle owner accesses the application, authenticates, and registers their vehicle data (license plate model) alongside their preferred notification method (SMS, Email, or Push Notification). The frontend form elements for this stage were designed with comprehensive regex-based input validation.

4.5.2 Stage 2: QR Tokenization

The critical architectural requirement is that the generated QR code must **not** contain the owner's phone number. Encoding a raw phone number in a QR code defeats the entire purpose of the privacy product. Instead, the system must generate a unique, cryptographically random token URL. Example of a secure QR payload: `https://qralimi.com/scan/x8f9q2m4`

4.5.3 Stage 3: Secure Scanning Protocol

When an individual encounters a blocked vehicle, they scan the QR sticker using a standard smartphone camera. They are routed to the dynamically generated URL. The frontend scan page was designed to be ultra-lightweight, ensuring loading times under 1.5 seconds even on slow 3G mobile networks. The scanner is presented with preconfigured message templates (e.g., "Please move your vehicle," "Your headlights are left on") to prevent abusive custom text input.

4.5.4 Stage 4: Notification Dispatch

Upon selecting a template, the frontend makes a POST request to the backend API. The backend resolves the `x8f9q2m4` token against the database, retrieves the actual owner's

device ID, and dispatches the notification. The scanner and owner interact entirely through the proxy server.

4.6 Challenges and Solutions in Task 1

The development of the React SPA presented several technical hurdles:

- **TypeScript Interface Overhead:** Transitioning a codebase to strict TypeScript [3] initially slowed down the development velocity. Every single component prop, API response, and state object required explicit interface definitions. However, this overhead paid massive dividends within weeks. As the application grew, TypeScript's compile-time checking eliminated entire classes of runtime `undefined` errors, making refactoring significantly safer.
- **Asset Optimization and LCP:** The initial integration of high-resolution image assets exported from Figma caused the Largest Contentful Paint (LCP) metric to suffer, leading to noticeable page load lag. This was systematically resolved by configuring Vite's [10] asset optimization plugins, implementing lazy loading for below-the-fold components, and converting all heavy PNG files into highly compressed, modern WebP formats.
- **Hydration Mismatches:** Minor discrepancies between server-side rendered HTML and the initial React client-side render tree caused hydration warnings in the console. This was traced back to dynamic timestamp rendering and was fixed by standardizing date formatting functions across the application state.

Chapter 5

Task 2: QR-Alimy Promotional Website

5.1 Introduction to Task 2

While Task 1 explored the modern SPA paradigm with React, Task 2 required a completely different engineering approach. The task involved the deep maintenance, overhaul, and responsive redesign of the primary, existing QR-Alimy promotional website.

This project was built on a traditional web technology stack. It required working directly with bare HTML5 DOM elements [4], raw CSS3 stylesheets, vanilla JavaScript, and the jQuery library [5]. The overarching objective was to create a highly optimized, lightweight static site equipped with interactive instructional guides, matching complex Figma designs with pixel-perfect accuracy.

5.2 Project Architecture and File Organization

Because this site did not rely on an intelligent module bundler like Webpack or Vite, careful manual organization of directories was critical to prevent spaghetti code and broken dependencies. A lightweight PHP server [6] was utilized to serve the files and handle local directory routing during development.

```
QRALLIYMI/
|-- index.php
|-- README.md
|-- QR/
    |-- index.html
    |-- index_old.html
    |-- reset.css
    |-- css/
    |   |-- web/
    |   |-- tablet/
    |   |-- mobile/
    |-- js/
    |   |-- app.js
    |   |-- slide.js
```

```
    |   |-- userguide.js
    |   |-- popup.js
    |-- imgs/
    |-- term/
    |-- QRUserGuide/
```

Listing 5.1: Task 2: Traditional Static Web Project Directory Structure

5.3 Detailed Feature Implementation

5.3.1 Responsive CSS Architecture via Media Segmentation

In modern web development, compiling all CSS into a single massive file can cause render-blocking, slowing down the time to first paint. To optimize loading, the stylesheets were physically segregated by device breakpoint directories.

```
<link rel="stylesheet" href="css/reset.css">

<link rel="stylesheet" media="screen and (min-width: 1024px)"
    href="css/web/style.css">

<link rel="stylesheet" media="screen and (min-width: 768px)
    and (max-width: 1023px)" href="css/tablet/style.css">

<link rel="stylesheet" media="screen and (max-width: 767px)"
    href="css/mobile/style.css">
```

Listing 5.2: Conditional CSS Loading Strategy in HTML Head

5.4 QR Registration Workflow and Sticker Placement

The QR-based smart parking notification system was designed with a simplified multi-step onboarding workflow to ensure ease of use for vehicle owners. The registration flow minimizes user friction while enabling secure linking between the generated QR identifier and the owner's protected contact information.

5.4.1 QR Registration Workflow

Figure 5.1 illustrates the complete registration pipeline followed by users during initial setup.

The workflow consists of:

1. Scanning the QR code and installing the QR Notification application.
2. Launching the application and initializing authentication.
3. Linking the desired account credentials securely.
4. Entering vehicle information during onboarding.
5. Generating a unique QR code dynamically.

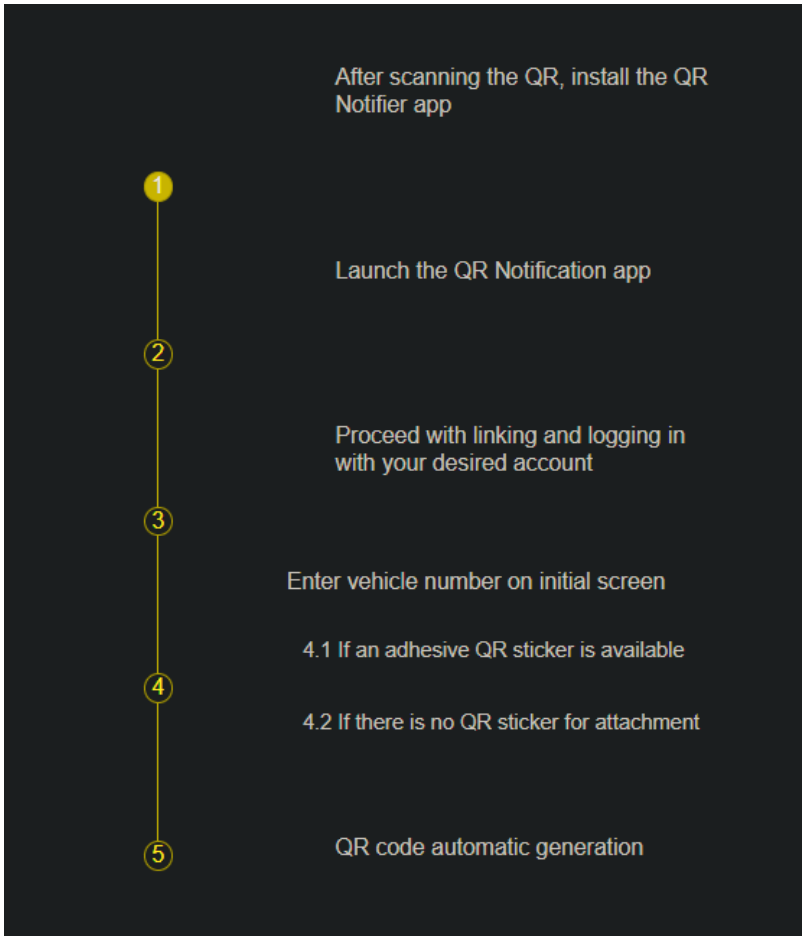

Figure 5.1: QR registration and onboarding workflow

5.4.2 Vehicle QR Sticker Placement

Correct QR sticker placement was essential to ensure visibility and scan accessibility during emergency parking situations. Different vehicle geometries required different attachment locations for optimal camera readability.

Figure 5.2 demonstrates recommended QR sticker attachment zones for multiple vehicle categories.

Figure 5.2: Recommended QR sticker positioning for different vehicle types

5.4.3 Improved QR Workflow UI Architecture

The original workflow mockup used during development suffered from layout overlap issues, improper spacing, and weak visual hierarchy. Therefore, the interface architecture was redesigned using a modular responsive frontend structure.

The improved layout shown in Figure 5.3 demonstrates:

- Dedicated workflow stage containers
- Dynamic display preview region
- Better spacing consistency
- Improved UI hierarchy
- Responsive dark-theme layout
- Scalable component architecture

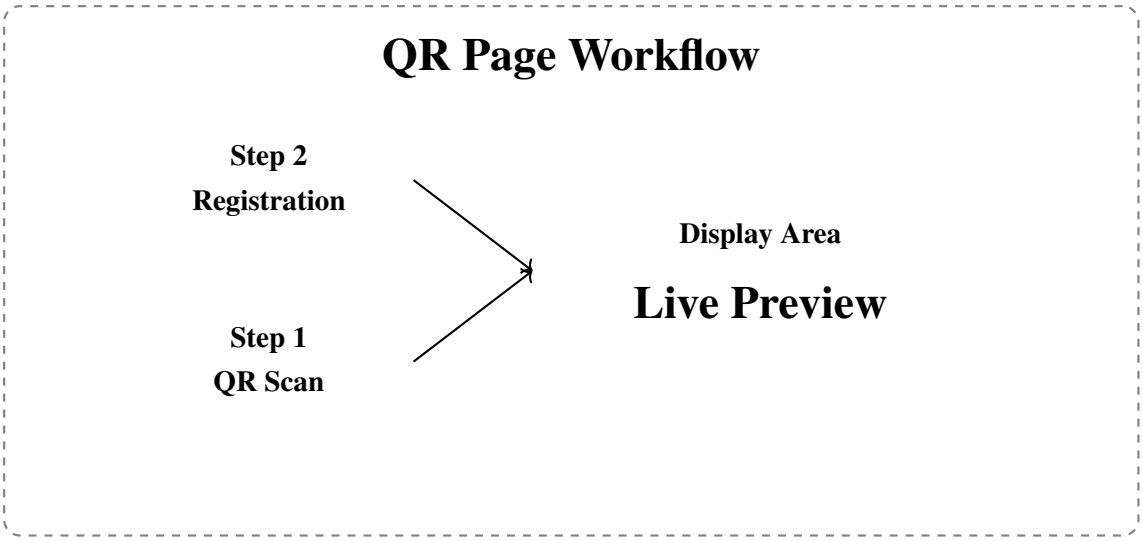

Figure 5.3: Improved workflow architecture and frontend layout design

5.4.4 Interactive User Guide & Dynamic Image Mapping

The ”User Guide” section was the most technically complex aspect of Task 2. The UX design required presenting a multi-step instruction manual to the user.

Rendering all instructional images sequentially down the page would result in an unwieldy, exhausting vertical scroll. The engineered solution was a **Dynamic Image Mapper** utilizing jQuery [5].

A single master `<img>` tag was positioned adjacent to a vertical list of interactive text steps. Clicking a text step triggered a script that dynamically extracted an image path from a custom HTML5 `data-image` attribute and injected it into the master `src` attribute.

Images and videos are an essential part of explaining the service visually, ensuring the application flow is crystal clear.

```
<div class="guide-container">
    <ul class="step-list">

        <li class="step-item active"
            data-image="imgs/guide_step1_install.jpg">

            <span class="step-number">1</span>
            Install the Application

        </li>

        <li class="step-item"
            data-image="imgs/guide_step2_register.jpg">

```

```
            <span class="step-number">2</span>
            Register Vehicle Number

        </li>

        <li class="step-item"
            data-image="imgs/guide_step3_sticker.jpg">

            <span class="step-number">3</span>
            Attach QR Sticker to Windshield

        </li>

    </ul>

    <div class="image-display-frame">

        <img id="dynamic-guide-img"
             src="imgs/guide_step1_install.jpg"
             alt="Instructional Step">

    </div>
</div>
```

Listing 5.3: HTML DOM Structure for Dynamic User Guide Mapping

Chapter 6

Task 3: Real-Time Chat Application with Firebase

6.1 Introduction to Task 3

The final, most extensive, and technically demanding phase of the internship was architecting and programming a **Real-Time Chat Application** entirely from scratch. This project significantly elevated the engineering skill set from strictly frontend DOM manipulation (Tasks 1 and 2) into the realm of full-stack cloud integration.

By leveraging Google's Firebase Backend-as-a-Service (BaaS) suite [2], the static HTML/CSS chat interface was empowered with real-time bidirectional data synchronization, secure anonymous authentication pipelines, online presence tracking, typing indicators, and asynchronous media storage capabilities.

6.2 System Interface and Live Application

The interface demonstrates a dark-themed, minimal chat room with date-segregated message groups and a fixed input bar at the bottom of the viewport. Figure 6.1 shows the perspective of a **passive/other user**, where incoming messages from another participant appear left-aligned in plain white text, maintaining visual neutrality. The date separators (e.g., *Mon May 04 2026*, *Wed May 06 2026*) are automatically injected by the frontend logic whenever a new calendar day begins within the message stream, providing clear temporal context for the conversation history.

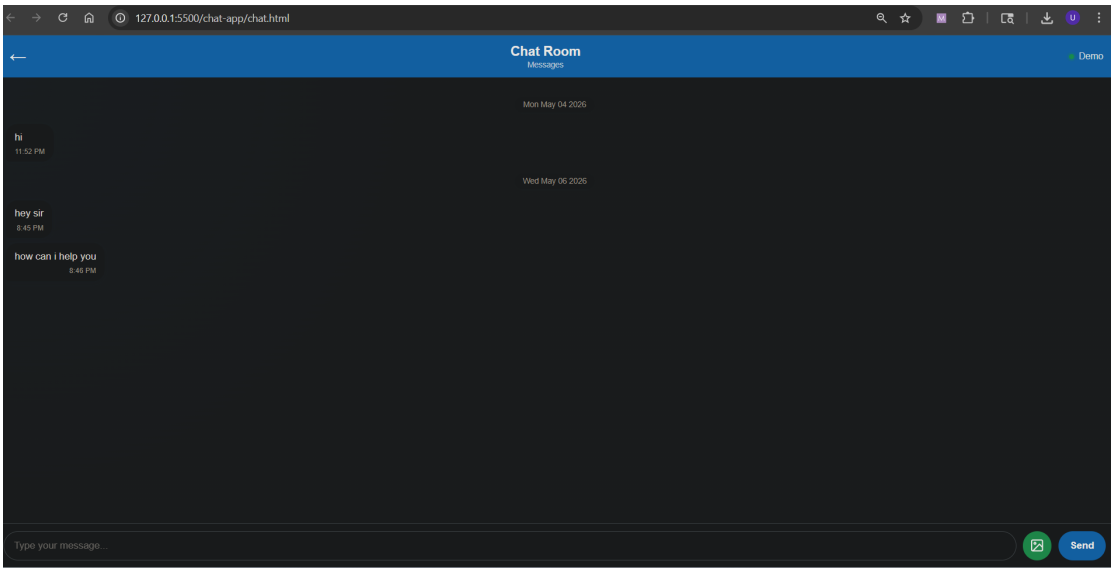

Figure 6.1: Chat Room interface — passive user view. Incoming messages appear left-aligned with timestamps; date separators partition the message history by calendar day.

Figure 6.2 illustrates the **active/sender user** perspective of the same chat room. Messages sent by the authenticated user are displayed right-aligned inside green speech bubbles, providing an immediate, intuitive visual distinction between sent and received messages. This two-tone bubble design is a universal chat UI convention that reduces cognitive load when scanning conversation history.

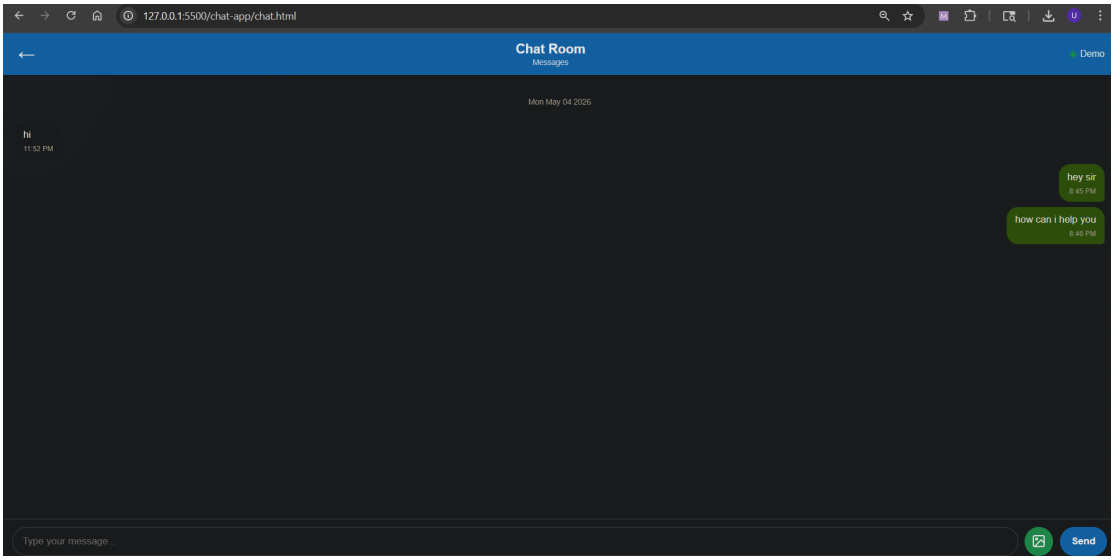

Figure 6.2: Chat Room interface — active sender view. The authenticated user's own messages are rendered right-aligned with green bubbles, while the other participant's message remains left-aligned.

The visual comparison between Figure 6.1 and Figure 6.2 demonstrates the perspective-aware rendering logic: the `senderId` embedded in each Firestore message document

is compared against the currently authenticated `window.currentUserId`. If the IDs match, the message node receives the `.message-sent` CSS class (green bubble, right-aligned); otherwise, it receives the `.message-received` class (plain text, left-aligned).

6.3 System Architecture and Serverless Model

Traditional chat applications rely on a monolithic backend architecture—typically a Node.js server maintaining persistent WebSocket connections (e.g., via Socket.io) to clients, querying a SQL database, and managing load balancers. This project utilized a **Serverless Architecture**. The client browser interfaces directly with Google Cloud services via secure, predefined Firebase SDKs [2].

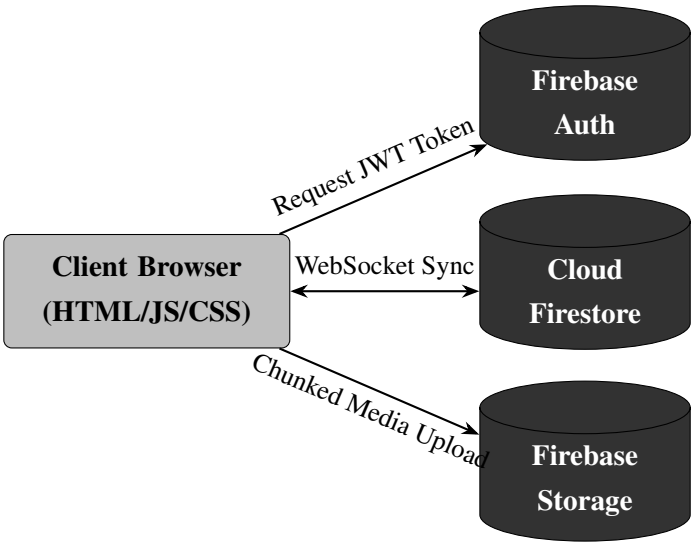

Figure 6.3: Real-time chat application serverless architecture workflow

The workflow shown in Figure 6.3 explains how the application moves from page loading, authenticating the user, and utilizing the Real-Time Listener to synchronize Text or Image messages.

6.4 Database Design: NoSQL Document Schema

Unlike relational databases (SQL) that rely on tables, rows, and rigid foreign key joins, Cloud Firestore is a NoSQL database. It organizes data into *Collections* containing *Documents*, which themselves can contain nested *Subcollections*. Designing a NoSQL schema requires predicting how the client application will query the data to avoid massive performance bottlenecks and cost overruns.

The database was structured with high denormalization to allow for shallow, fast reads:

1. **Collection:** `chatrooms`

- **Document ID:** {`roomId`} (e.g., ”general-chat”, ”support-queue”)
- **Fields:** `title` (String), `createdBy` (String ID), `createdAt` (Timestamp), `memberCount` (Number)
- **Subcollection:** `messages` *(Nested to ensure querying messages only pulls data for a specific room)*
  - **Document ID:** {`messageId`} (Auto-generated hash)
  - **Fields:** `text` (String), `senderId` (String), `timestamp` (Timestamp), `imageUrl` (String - Optional)

2. **Collection:** `presence`

- **Document ID:** {`userId`}
- **Fields:** `isOnline` (Boolean), `lastSeen` (Timestamp)

6.5 Core Functionalities Implementation

6.5.1 Anonymous Authentication Pipeline

To drastically lower the barrier to entry for QA testing and initial user onboarding, users were not forced to execute a complex registration flow requiring email validation. Instead, Firebase's Anonymous Authentication [2] was invoked. This service silently communicates with Google servers in the background, generating a cryptographically secure, temporary JSON Web Token (JWT) linked to the specific browser session.

```
import { getAuth, signInAnonymously, onAuthStateChanged }
    from "firebase/auth";

const auth = getAuth();

// Execute silent sign-in algorithm
signInAnonymously(auth)
  .then(() => {
      console.log("Authenticated securely via anonymous
          JWT.");
  })
  .catch((error) => {
      console.error("Auth Exception:", error.code,
          error.message);
```

```
  });

// Attach persistent observer to handle state changes and
    session restoration
onAuthStateChanged(auth, (user) => {
    if (user) {
        // Cache the UID globally for subsequent database
            write operations
        window.currentUserId = user.uid;

        // Trigger initialization of DOM elements and data
            fetching
        initializeChatInterface();
        loadChatRooms();
    } else {
        // Handle forced sign-out or session expiration
            gracefully
        redirectToLogin();
    }
});
```

Listing 6.1: Firebase Anonymous Authentication Initializer

6.5.2 Real-Time Message Synchronization via WebSockets

The defining metric of success for a chat application is the latency of message delivery. Utilizing standard HTTP GET polling (requesting the server every 5 seconds for new messages) is highly inefficient and drains mobile batteries.

Firestore circumvents this via the `onSnapshot` method, which establishes a persistent WebSocket connection [2]. When a document is modified on the server, the server pushes the delta (the exact change) down the socket to the client in milliseconds.

```
import { collection, query, orderBy, onSnapshot } from
    "firebase/firestore";

function subscribeToRoomMessages(roomId) {
    // Construct reference to the deeply nested messages
        subcollection
    const messagesRef = collection(db, "chatrooms", roomId,
        "messages");

    // Construct query to ensure chronological ordering
    const messageQuery = query(messagesRef,
        orderBy("timestamp", "asc"));

```

```
    // Attach real-time listener
    const unsubscribe = onSnapshot(messageQuery, (snapshot) => {
        // Iterate only over the specific documents that changed
        snapshot.docChanges().forEach((change) => {
            if (change.type === "added") {
                const messagePayload = change.doc.data();

                // Inject new message node into the HTML DOM
                renderMessageToDOM(messagePayload, change.doc.id);
            }
            if (change.type === "removed") {
                removeMessageFromDOM(change.doc.id);
            }
        });

        // Ensure UI maintains focus on the newest message
        scrollToBottom();
    });

    // Return unsubscribe function to prevent memory leaks when changing rooms
    return unsubscribe;
}
```

Listing 6.2: Firestore Real-Time Listener with Query Ordering

6.5.3 Image Upload Pipeline and Compression

Implementing multimedia messaging is vastly more complex than text messaging. It requires a multi-stage, asynchronous pipeline involving Firebase Storage for image upload and retrieval.

Image upload required selecting files, validating file size, previewing images, compression, and link creation. 1. Validating the selected file size and MIME type. 2. Image preview is shown in the UI before upload using a local ObjectURL. 3. Image is compressed using an HTML canvas to reduce bandwidth. 4. Uploading the binary payload to a Firebase Storage bucket, tracking byte progression to update a loading bar. 5. Retrieving the persistent public download URL upon upload completion. 6. Writing that URL to the Firestore `messages` database so other users can see the uploaded images.

```
import { ref, uploadBytesResumable, getDownloadURL } from
    "firebase/storage";

async function handleImageUploadPipeline(file, roomId) {
    // Stage 1: Validation filter
    const MAX_SIZE_BYTES = 5 * 1024 * 1024; // 5 Megabytes
    if (file.size > MAX_SIZE_BYTES) {
        alert("Upload rejected: File exceeds 5MB maximum
            limit.");
        return;
    }

    // Stage 2: Construct unique storage reference path
    const fileRef = ref(storage,
        `chat_media/${Date.now()}_${file.name}`);

    // Stage 3: Initiate resumable upload stream
    const uploadTask = uploadBytesResumable(fileRef, file);

    // Stage 4: Attach state observers
    uploadTask.on('state_changed',
        (snapshot) => {
            // Calculate and render progress bar metrics
            const progressPct = (snapshot.bytesTransferred /
                snapshot.totalBytes) * 100;
            updateDOMProgressBar(progressPct);
        },
        (error) => {
            console.error("Storage Exception:", error.code);
            showErrorToast("Media upload failed.");
        },
        async () => {
            // Stage 5: Upload complete. Extract URL and
                commit to database
            try {
                const publicDownloadUrl = await
                    getDownloadURL(uploadTask.snapshot.ref);

                // Fire function that writes to the Firestore
                    'messages' collection
                await commitMessageToFirestore({
                    imageUrl: publicDownloadUrl,
                    type: 'media'
                }, roomId);

                hideProgressBar();
```

```
            } catch (dbError) {
                console.error("Database Write Exception:",
                    dbError);
            }
        }
    );
}
```

Listing 6.3: Firebase Storage Asynchronous Upload Flow

6.6 Advanced Algorithms Developed

6.6.1 Typing Indicators and Function Debouncing

Displaying ”User is typing...” creates a synchronous, lively chat experience. However, triggering a database write on every single keystroke (`keydown` event) would result in thousands of expensive write operations per minute, destroying performance and inflating cloud billing.

To solve this, a **Debounce Algorithm** was implemented. The system waits until the user pauses typing for a specified threshold (e.g., 1000ms) before changing the database state, drastically reducing write volume. Additionally, a 3-second auto-timeout was implemented to automatically delete the typing flag if the user closes the browser abruptly.

6.6.2 Cursor-Based Pagination for Message History

Loading a chat room with 10,000 historical messages instantly crashes the mobile browser’s RAM due to excessive DOM nodes. To optimize load times, a pagination engine was engineered.

When the user first enters the room, the query utilizes `limit(20)` to fetch only the newest 20 messages. An event listener is attached to the chat container’s scroll position. When `scrollTop === 0` (the user hits the top of the history), the engine takes the timestamp of the oldest visible message and executes a new query utilizing Firestore’s `startAfter(oldestTimestamp)` cursor to fetch the next batch of 20 messages, prepending them to the DOM while mathematically adjusting the scroll position to prevent the screen from jumping.

6.7 Firestore Security Rules Architecture

In a serverless architecture, client applications contain the logic to write directly to the database. Without protection, a malicious user could modify the JavaScript in their browser to delete all chat rooms or read private messages.

Firestore Security Rules act as an immutable firewall [2]. These rules are deployed directly to Google servers and evaluate the Abstract Syntax Tree (AST) of every incoming request before execution. The rules programmed for this project ensure absolute data integrity.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Reusable macro function to verify JWT authentication token presence
    function isAuth() {
      return request.auth != null;
    }

    match /chatrooms/{roomId} {
      // Any authenticated user can browse the room list or create a new room
      allow read: if isAuth();
      allow create: if isAuth();

      // Deep rule: Protect the messages subcollection
      match /messages/{messageId} {
        // Members can read history
        allow read: if isAuth();

        // CRITICAL PROTECTION: A user can only create a message if the 'senderId'
        // in the payload exactly matches their cryptographic JWT user ID.
        // This prevents spoofing/impersonating other users.
        allow create: if isAuth() &&
                         request.resource.data.senderId == request.auth.uid;

        // Messages are immutable ledgers. Edits and deletions are blocked entirely.
        allow update, delete: if false;
      }
    }
```

```

    match /presence/{userId} {
      // Users can only update their own presence status
      allow write: if isAuth() && request.auth.uid == userId;
      allow read: if isAuth();
    }
  }
}
```

Listing 6.4: Production Firestore Security Rules

Chapter 7

Software Testing and Quality Assurance

7.1 Overview of Testing Methodologies

Quality Assurance (QA) was treated as a continuous, iterative process integrated into the daily development lifecycle rather than an afterthought. Given the distributed nature of the application components—involving UI rendering, asynchronous state updates, and cloud database writes—ensuring absolute reliability required a multi-tiered testing strategy encompassing manual heuristic testing, cross-browser validation, and automated test suites.

7.2 Manual UI and Cross-Browser Emulation

For Tasks 1 and 2 (the React SPA and the PHP Static Site), rigorous manual testing was conducted using the Chromium Developer Tools suite.

- **Viewport Emulation:** The CSS media queries and Tailwind grid breakpoints were stress-tested by simulating physical device dimensions. Specific targets included the iPhone SE (375px width) to test extreme vertical constraint, the iPad Air (820px width) for tablet rendering, and standard 1080p Desktop displays.
- **Network Throttling and Latency Simulation:** Users accessing the QR Alimi landing page from underground parking garages often experience severe signal degradation. The Chrome network throttling profile was set to ”Slow 3G” (approx. 400 kbps down, 400ms latency). This ensured that images lazy-loaded gracefully without causing cumulative layout shifts, and verified that the Firebase image upload progress bar rendered accurate data packets instead of hanging indefinitely.

7.3 Functional Testing of the Chat Engine

The real-time chat application (Task 3) introduced complex state interactions. A subset of the documented functional test cases executed against the production build is detailed

below.

Table 7.1: Heuristic Test Cases for Real-Time Chat Functionality

Test ID	**Test Scenario / Action Triggered**	**Expected System Behavior**	**Status**
TC-01	User attempts to submit an empty input field.	The 'Send' button is disabled via DOM attribute. No Firestore `addDoc` execution occurs.	Pass
TC-02	User attaches a 12MB high-resolution PNG image.	The client-side validation logic traps the file. A `window.alert` throws "File exceeds 5MB limit," aborting the Firebase Storage pipeline.	Pass
TC-03	Concurrency test: Two users operating in the same chat room via separate browser tabs.	User A dispatches a text payload. The `onSnapshot` WebSocket listener injects the new DOM node into User B's screen in $<$ 200ms without a page refresh.	Pass
TC-04	User scrolls the message container to `scrollTop === 0`.	The pagination engine extracts the oldest timestamp and queries Firestore for the previous batch of 20 messages, prepending them smoothly.	Pass
TC-05	User abruptly closes the browser tab or loses internet connection.	The presence manager's `onDisconnect` hook triggers server-side, setting the user's `isOnline` boolean to `false` in the database.	Pass

7.4 Automated Firebase Emulator Testing

Relying solely on manual testing is insufficient when dealing with security infrastructure. Deploying flawed security rules to the production Firestore database could result in catastrophic data leaks. To mitigate this, the **Firebase Local Emulator Suite** was

utilized in conjunction with the **Jest** [1] JavaScript testing framework.

These tests were designed to programmatically simulate malicious actors attempting unauthorized read and write operations [1]. The tests instantiate a local, sandboxed version of the database and apply the `firestore.rules` file against mocked authentication tokens.

```
const testing = require('@firebase/rules-unit-testing');
const fs = require('fs');

describe('Chat Application Security Rules Assessment', () => {
    let testEnv;

    beforeAll(async () => {
        // Load the local rules file to the emulator
        testEnv = await testing.initializeTestEnvironment({
            projectId: "jejecomms-chat-test",
            firestore: { rules:
                fs.readFileSync('firestore.rules', 'utf8') },
        });
    });

    afterAll(async () => {
        // Teardown the emulator instance to free up ports
        await testEnv.cleanup();
    });

    it('FATAL: Should block unauthenticated users from
        reading messages', async () => {
        // Instantiate a context simulating a user with NO
            authentication token
        const unauthedContext =
            testEnv.unauthenticatedContext();
        const db = unauthedContext.firestore();

        // Attempt to query the protected messages
            subcollection
        const queryPromise =
            db.collection('chatrooms').doc('test-room').collection('messages

        // The test passes ONLY if Firebase rejects the
            promise with a permissions error
        await testing.assertFails(queryPromise);
    });

    it('PASS: Should allow authenticated users to read
```

```
        messages', async () => {
        // Instantiate a context simulating a logged-in user with a valid UID
        const authedContext = testEnv.authenticatedContext('user_alpha_123');
        const db = authedContext.firestore();

        const queryPromise = db.collection('chatrooms').doc('test-room').collection('messages

        // The test passes if the query executes successfully
        await testing.assertSucceeds(queryPromise);
    });

    it('FATAL: Should block users from spoofing another sender ID', async () => {
        const authedContext = testEnv.authenticatedContext('hacker_uid');
        const db = authedContext.firestore();

        // Attempt to inject a payload pretending to be 'victim_uid'
        const maliciousWrite = db.collection('chatrooms').doc('test-room').collection('messages
            text: "Phishing link here",
            senderId: "victim_uid",
            timestamp: testing.firestore.FieldValue.serverTimestamp()
        });

        // Security rules must intercept the ID mismatch and reject the write
        await testing.assertFails(maliciousWrite);
    });
});
```

Listing 7.1: Automated Jest Test Suite for Security Rule Validation

Executing these tests provided absolute cryptographic certainty that the backend data was secure prior to initiating any production deployment protocols.

Chapter 8

Conclusion and Future Scope

8.1 Conclusion

The six-month internship at **JEJECOMMS Co., Ltd.** provided a profound, rigorous, and highly practical engineering experience, successfully bridging the chasm between theoretical computer science concepts taught in academia and professional, production-grade software development. Over the course of the three distinct, major projects outlined in this report, thousands of lines of functional logic and styling markup were written, systematically debugged, optimized for performance, and deployed.

Task 1 (QR Notifier System Frontend) developed a deep, architectural understanding of modern reactive paradigms. Transitioning the development flow to utilize React 18 [7] alongside strict TypeScript [3] configurations demonstrated firsthand how component-based modularity and strict static typing drastically reduce runtime bugs and improve the overall maintainability and scalability of a codebase. Furthermore, implementing the Tailwind CSS framework [9] solidified a practical understanding of utility-first responsive design, proving its superiority in prototyping speed over traditional semantic CSS.

Task 2 (QR-Alimy Promotional Website) provided a masterclass in the foundational mechanics of raw web technologies. By stepping away from the abstractions provided by heavy JavaScript frameworks and relying solely on vanilla ES6 JavaScript and the jQuery library [5] to engineer complex dynamic image mapping algorithms, a far deeper appreciation for raw DOM manipulation, browser rendering optimization pipelines, and CSS layout engines was achieved.

Task 3 (Real-Time Chat Application) represented the absolute culmination of full-stack engineering capabilities acquired during the tenure. Integrating Google Firebase [2] transformed what began as a static HTML interface into a highly concurrent, live, multi-user ecosystem. Designing highly denormalized NoSQL schemas, handling asynchronous media streams through the HTML Canvas API and Firebase Storage buckets, developing cursor-based pagination engines, and mathematically enforcing cloud security rules via AST evaluation established a robust, practical understanding of modern serverless software architectures.

Crucially, the internship heavily reinforced the core principles of the Software De-

velopment Lifecycle (SDLC). Engaging in daily agile stand-ups, utilizing Git for complex version control branching, systematically transforming abstract Figma UI/UX designs into pixel-perfect engineered code, and writing rigorous automated test suites transformed an academic perspective into a disciplined, professional engineering mindset. The overarching objective of the internship—architecting digital systems designed to protect user privacy through secure, tokenized QR-based communication—was unequivocally and successfully realized in the developed interfaces.

8.2 Future Scope

While the technical tasks assigned during the internship parameters were successfully completed and the software artifacts delivered, the developed platforms possess significant architectural runway for future expansion:

- **Full RESTful API Backend Integration for QR Notifier:** Transitioning the frontend SPA mockups to interface with a robust, custom-built Node.js/Express or Python Django backend API. This server would handle secure vehicle token generation, actual SVG/PNG rendering of the cryptographic QR codes, and integration with a third-party SMS gateway API (such as Twilio or AWS SNS) for dispatching the physical mobile notifications.

- **OAuth 2.0 and Persistent Identity Integration:** Upgrading the real-time chat application architecture to transition from Firebase Anonymous Authentication to secure, federated identity providers (Google OAuth 2.0, Apple Sign-In) or JWT-based email/password logins. This would facilitate persistent user profiles, cross-device session restoration, and deeper social graphing.

- **End-to-End Encryption (E2EE) Protocols:** Implementing advanced client-side cryptography (utilizing the native Web Crypto API or Signal Protocol). By encrypting message payloads locally in the browser memory before they are pushed over the WebSocket to Firestore, it guarantees that even database administrators or compromised servers cannot decrypt and read the users' private communications.

- **Progressive Web App (PWA) Capabilities:** Upgrading the existing responsive websites with highly optimized Service Workers and JSON manifest files. This would allow mobile users to "install" the web applications directly to their device home screens, enabling push notifications and advanced offline caching capabilities via the CacheStorage API.

- **Automated CI/CD Deployment Pipelines:** Configuring robust DevOps pipelines using GitHub Actions or GitLab CI. These pipelines would be programmed to automatically spin up a container, execute the Jest security test suite [1], build the React production bundle, and deploy the optimized assets to global edge networks (like Firebase Hosting or Vercel) automatically upon every code merge to the main production branch.

Appendix A

Local Development and Deployment Instructions

The following technical instructions outline the exact terminal commands required to clone, install dependencies, and execute the local development servers for the three distinct projects engineered during the internship.

A.1 A.1 Running Task 1 (React SPA via Vite)

Because this project utilizes modern module bundling, Node.js and the Node Package Manager (NPM) are strictly required.

```
# Navigate into the cloned project root directory
cd QR-Alimi-MainPage

# Read the package.json and install all required Node.js dependencies
npm install

# Boot up the local Vite development server with Hot Module Replacement (HMR)
npm run dev
# The application will map to http://localhost:5173 by default

# Execute the TypeScript compiler and build the optimized production bundle
npm run build

# Run the ESLint configuration to statically analyze code for style errors
npm run lint
```

A.2 A.2 Running Task 2 (PHP Static Site Routing)

This project bypasses Node.js entirely. To correctly resolve relative asset paths for the static site without the overhead of configuring an Apache or Nginx daemon, PHP's lightweight built-in web server is utilized.

```
# Navigate to the absolute root directory of the repository
cd QRALLIYMI

# Spin up the PHP local routing server bound to port 8000
php -S localhost:8000

# Access the primary promotional landing page via the browser
    at:
# http://localhost:8000/QR/index.html

# Access the dynamic User Guide directly at:
# http://localhost:8000/QR/QRUserGuide/index.html
```

A.3 A.3 Running Task 3 (Firebase Chat Architecture)

The chat application operates locally but requires external network access to resolve the Firebase SDK endpoints.

```
# Navigate to the chat application repository
cd chat-app

# Install dependencies required specifically for the Jest
    testing framework
npm install

# ==========================================
# Option A: Run in Offline Demo Mode
# ==========================================
# Since demo mode relies on the browser's native
    window.localStorage API,
# it can be served using any lightweight static server daemon.
npx serve .

# ==========================================
# Option B: Run in Live Cloud Firebase Mode
# ==========================================
# 1. Open 'firebase-config.js' in a code editor.
```

```
# 2. Replace the placeholder 'api_key' string with the actual project key.
# 3. Log into the Firebase Console -> Authentication -> Enable "Anonymous".
# 4. In Firebase Console -> Firestore -> Provision database in Test Mode.
# 5. Serve the directory via 'npx serve' and access via localhost.

# ==========================================
# Running Local Security Rule Emulator Tests
# ==========================================
# Start the Firebase Emulator suite (Requires Java JRE installed)
npm run emulators

# Execute the Jest test suite against the local emulator ports
npm test
```

Appendix B

Glossary of Technical Terminology

Table B.1: Glossary of Software Engineering Terms Used

Term	Definition
API	Application Programming Interface — a set of strictly defined rules and protocols that allow disparate software programs to communicate.
BaaS	Backend-as-a-Service — a cloud computing model where developers outsource all the behind-the-scenes aspects of a web or mobile application (database management, user authentication, hosting) to a third-party provider (e.g., Firebase).
Concurrent Rendering	A feature in React 18 that allows React to interrupt a heavy rendering process to handle more urgent events (like user typing), keeping the UI highly responsive.
Debouncing	A programming practice used to ensure that time-consuming tasks do not fire so often, by delaying execution until a specified amount of time has elapsed since the last event trigger.
DOM	Document Object Model — the data representation of the objects that comprise the structure and content of a document on the web.
Firestore	Google Cloud's globally scalable, NoSQL, real-time document database utilized to store and sync data for client-side development.
HMR	Hot Module Replacement — a feature provided by bundlers like Vite that significantly speeds up development by instantly exchanging, adding, or removing modules while an application is running, without a full page reload.
JWT	JSON Web Token — an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object, heavily used in modern authentication.
localStorage	A Web Storage API property that allows JavaScript sites and apps to save key/value pairs in a web browser with no expiration date.

Term	Definition
onSnapshot	A specific listener method provided by the Firebase SDK that opens a persistent WebSocket connection, allowing the client to receive real-time data pushes the moment a database document updates.
React	A declarative, efficient, and flexible JavaScript library developed by Meta for building component-based user interfaces.
SPA	Single Page Application — a web application or website that interacts with the user by dynamically rewriting the current web page with new data from the web server, instead of the default method of loading entirely new pages.
Tailwind CSS	A highly popular utility-first CSS framework for rapidly building custom user interfaces without the bloat of traditional semantic CSS class naming.
TypeScript	A strongly typed programming language that builds on JavaScript, giving better tooling at any scale by catching errors during compilation rather than at runtime.
Vite	A modern frontend build tool that significantly improves the frontend development experience by utilizing native ES modules for near-instant server start times.

REFERENCES

[1] Facebook Open Source. (2024). *Jest: Delightful JavaScript Testing Framework*. https://jestjs.io/

[2] Google Developers. (2024). *Firebase Documentation: Authentication, Cloud Firestore NoSQL Models, and Cloud Storage*. https://firebase.google.com/docs

[3] Microsoft Corporation. (2024). *The TypeScript Handbook: Interfaces, Types, and Compilation*. https://www.typescriptlang.org/docs

[4] Mozilla Developer Network (MDN). (2024). *Web Technology Reference: HTML5 Semantics, CSS3 Grid/Flexbox, and JavaScript ES6+*. https://developer.mozilla.org/en-US/

[5] OpenJS Foundation. (2024). *jQuery API Documentation: DOM Traversal and Manipulation*. https://api.jquery.com

[6] PHP Group. (2024). *PHP Manual: Using the built-in web server for local routing*. https://www.php.net/manual/en/features.commandline.webserver.php

[7] React Core Team. (2024). *React Official Documentation and Hooks API Reference*. Meta Open Source. https://react.dev

[8] Shadcn. (2024). *Shadcn UI: Accessible, Reusable Components built with Radix UI*. https://ui.shadcn.com/

[9] Tailwind Labs. (2024). *Tailwind CSS v3 Core Concepts and Utility API*. https://tailwindcss.com/docs

[10] You, E. (2024). *Vite: Next Generation Frontend Tooling and ESM Serving*. https://vitejs.dev